Subtype entailment for bottom and binary product: arbitrary and finite trees

Author and affiliation placeholders

20 September 2026

Abstract

We give a terminating decision procedure for entailment between finite type expressions built from variables, bottom, and one covariant binary product. Variables range over arbitrary trees, including infinite nonregular trees. The proof reduces failure of entailment to a finite automaton problem with constraints on agreeing suffixes. The automata obtained from subtyping have an absorption property that separates this problem into a terminal case and an infinite-continuation case. The latter admits a finite periodic monitor. For the terminal case, we reverse a candidate word and use its border rank. Although reference words can grow without a known useful length bound, their first rank milestones form a bounded sequence of first returns. A first return adds a directed strip with one entrance to a coloured prefix graph. We give a finite construction that pulls a whole existential regular-path query back through this extension. Products of observers over each physical interval preserve the single shared generated word. Iterating this construction a bounded number of times and evaluating the resulting query on two-vertex graphs decides the terminal case. We prove soundness, completeness and termination of both branches and compose them with exact expression normalization. The decision procedure and its correctness theorem are also formalized in Lean. The construction does not assume a numerical witness-length bound. We also decide entailment over finite trees. A consuming cycle in a finite reverse-pop automaton rules out finite models; without such a cycle, every failed query has a finite canonical countermodel. This dichotomy gives a second executable decision theorem, also formalized in Lean.

Contents

Abstract **1**

1. Introduction and main result **3**
1.1 Relation to previous work . . . 3
1.2 Proof strategy . . . 4
1.3 Semantics and theorem . . . 4
1.4 Examples . . . 5

2. Expression normalization **5**

3. Canonical propagation and finite stack summaries **6**
3.1 The least model generated by seeds . . . 6
3.2 Effective pushdown reachability . . . 7
3.3 One finite flagged automaton for all separating words . . . 7

4. Absorption and the terminal/cyclic decomposition **8**

5. Reversal, border rank, and first-return milestones **9**

6. Prefix graphs and the exact first-return extension **10**
6.1 A finite coloured graph for a concrete reference . . . 10
6.2 First-return generation . . . 11
6.3 Example: an overlapping return . . . 12

7. A finite preimage construction for directed path queries **12**
7.1 The query language . . . 12
7.2 Finite diagram enumeration . . . 13
7.3 The old-core preimage . . . 14
7.4 Marks and portal edges . . . 15
7.5 One common generator on every physical interval . . . 15
7.6 Assembly of the finite query . . . 16
7.7 Proof of coverage: a promoted model gives a base model . . . 16
7.8 Proof of reconstruction: a base model gives one promoted model . . . 17
7.9 Example: two simultaneous observations . . . 17
7.10 Shared-word consistency . . . 18
7.11 The directedness restriction . . . 18

8. Completion without another return **18**

9. Terminal decision by finite preimage iteration **18**

10. Decision of infinite continuation **19**
10.1 The periodic suffix criterion . . . 20
10.2 A complete finite reverse monitor . . . 20
10.3 Return from an infinite stream to a finite witness . . . 21

11. The entailment decision procedure **21**
11.1 A uniform computable witness bound . . . 21

12. Comparison with restricted entailment results **22**
12.1 The constant-free fragment . . . 22
12.2 Effective regular countermodels . . . 23
12.3 Scope of the contribution . . . 24

13. Entailment over finite trees **24**
13.1 The finite growth test . . . 24
13.2 Finite canonical replacements and the decision . . . 25
13.3 Examples and scope . . . 26

14. Formalization and further questions **27**

References **28**

1. Introduction and main result

Subtype entailment asks whether every assignment satisfying a finite set of type inequalities also satisfies a specified inequality. The distinction between satisfiability and entailment is substantial: a constraint solver may find some consistent assignment without deciding what holds in all of them. Recursive types add a second difficulty, because a counterexample may require an infinite tree even though the input expressions are finite.

We study the signature consisting of bottom and one covariant binary product. Variables range over all trees in this signature, with no regularity restriction. The main result is a terminating decision procedure for the full finite-expression entailment problem in this domain. Its central ingredient is a finite preimage construction for existential regular-path queries on coloured prefix graphs. It eliminates a shared, unbounded first-return word while preserving every path constraint in a conjunction.

A further finite-control argument treats finite-tree valuations. A cycle that consumes an address letter forces unbounded product growth. When no such cycle exists, the same canonical construction produces finite countermodels. This gives a finite-tree decision without identifying finite and arbitrary-tree entailment in cases where they differ.

1.1 Relation to previous work

Henglein and Rehof established two important complexity landmarks for subtype entailment. For structural finite simple types over a nontrivial lattice of base types, they proved coNP-completeness; their hardness construction already uses a binary covariant constructor [1, Theorems 4.7–4.8 and Corollary 4.9]. They subsequently introduced constraint automata for nonstructural entailment and proved PSPACE-hardness for infinite trees [2, Theorem 1] and finite trees [2, Theorem 2] over a signature with bottom, top, product and function constructors. The same paper proves PSPACE-completeness for structural infinite-tree entailment over a lattice of constants [2, Theorem 5]. These results established the need to distinguish structural from nonstructural subtyping, and finite from infinite semantics, in both algorithms and complexity claims.

Our order is nonstructural: bottom lies below products of arbitrary shape. The variable domain, however, has no explicit top-labelled leaf. The presence of a greatest infinite product tree does not identify these two signatures. In particular, a lower-bound construction whose constraints mention only products can still use top in its valuations. The Henglein–Rehof hardness theorems therefore require a separate semantic transfer before they can be claimed as lower bounds for the exact domain studied here.

Niehren and Priesnitz developed the constraint-automaton approach into a characterization by restricted cap-automaton universality and a shared word equation $\pi\nu = \mu\pi$ [3, Theorems 1, 2, 4 and Lemma 1]. Their ambient signature contains one covariant constructor of positive arity, bottom and explicit top, and they distinguish finite, regular and arbitrary trees. This characterization exposes the repeated-word condition; it does not by itself provide a general universality algorithm. Their earlier work introduces P-automata and proves PSPACE-completeness for a restricted syntax of product equations and variable inequalities, over both arbitrary and finite bottom/product/top trees [4, Theorem 14]. Bottom and top are excluded from that input grammar, but remain available in valuations. Section 12 gives an explicit reduction of our constant-free fragment to that decidable case.

The starting top-free separation characterization was developed in an unpublished memo [5, Problem 1.7 and Theorem 2.6]. We use its product-address semantics and canonical propagation route. Address languages, constraint automata and periodic continuations are inherited ideas.

The additional construction here is the whole-conjunction first-return preimage of Section 7 and its bounded iteration in Section 9. Together with the absorbing summary and the periodic case, it yields a complete decision procedure for the stated domain.

1.2 Proof strategy

Failure of entailment has a finite separating address. A least propagation closure determines whether that address is consistent with all premises. Finite stack summaries turn the closure conditions into a deterministic word automaton with forbidden ordered pairs of cuts. A pair matters precisely when the suffix at its later cut agrees with the beginning of the earlier suffix; the difficulty is this repeated-word condition, not ordinary finite-state reachability.

The derived automata satisfy an absorption property. After an accepting state, a safe continuation has no new incoming forbidden pairs. This separates two cases. A safe cyclic continuation is decided by a finite reverse monitor. Otherwise, it suffices to consider an accepting state with no safe successor.

For the terminal case, reverse the word and count the nested borders of its prefixes. The number of states bounds this rank, although it need not bound the word's length. Successive first rank milestones are first returns of a reference prefix. Each such return adds a directed path with one entrance to its prefix graph. A whole query can be pulled back through this addition by finitely many diagrams. All observations of one interval use a common product observer, ensuring that they describe one generated word. A bounded number of these eliminations reduces the question to finite queries on two-vertex graphs. Sections 5–9 give the construction and prove both directions.

1.3 Semantics and theorem

Let V be a finite set of variables and let expressions be generated by

$$e ::= x \in V \mid \bot \mid e \times e.$$

Write $\Sigma = \{0, 1\}$ and ϵ for the empty word. Concatenation is juxtaposition. For $0 \le i \le |w|$, write $w[: i]$ for the prefix of length i and $w[i :]$ for the remaining suffix. A type is a prefix-closed set $X \subseteq \Sigma^*$ of **product-labelled addresses**. The empty set represents bottom. A product is

$$X \times Y = \{\epsilon\} \cup \{0u : u \in X\} \cup \{1u : u \in Y\}.$$

Subtyping is inclusion. A valuation $\rho : V \to \mathcal{T}$ extends to expressions by these operations. Given a finite list Γ of inequalities,

$$\Gamma \vDash s \le t \quad \iff \quad \forall \rho \, [\, (\forall (u \le v) \in \Gamma, [\![u]\!]_\rho \subseteq [\![v]\!]_\rho) \Rightarrow [\![s]\!]_\rho \subseteq [\![t]\!]_\rho \,].$$

There is no computability, regularity, finite-depth, or decidable-membership restriction on ρ. The query $s \le t$ is not inserted into Γ. Inconsistent constraints entail every query.

Theorem 1.1 (decidability). There is a total executable Boolean procedure D such that, for every finite Γ, s, t,

$$D(\Gamma, s, t) = \mathsf{true} \quad \iff \quad \Gamma \vDash s \le t.$$

Variables can be indexed by a finite initial segment of the natural numbers. Thus the input is an effective finite object; repeated premises do not change its interpretation.

A tree X is **finite** if its product-address set is finite, equivalently if there is an h such that $|u| < h$ for every $u \in X$. Write $\Gamma \vDash_{\mathrm{fin}} s \le t$ for the same universal condition restricted to valuations assigning finite trees to all variables.

Theorem 1.2 (finite-tree decidability). There is a total executable Boolean procedure D_{fin} such that, for every finite Γ, s, t,

$$D_{\mathrm{fin}}(\Gamma, s, t) = \mathsf{true} \quad \Longleftrightarrow \quad \Gamma \vDash_{\mathrm{fin}} s \le t.$$

Section 13 proves this theorem by an effective reduction to Theorem 1.1. Neither result requires a satisfiability promise.

The address-set representation includes the ordinary labelled-tree boundary conditions. The root exists even for bottom; a nonroot address ua exists exactly when u is a product address. All defined nonproduct nodes are bottom. Conversely, a rooted bottom/product diagram in which exactly product nodes have two children determines a prefix-closed product set. These two descriptions agree. The full set Σ^* is a greatest type in this domain, but it is an infinite tree of products, not an explicit top-labelled leaf. Adding such a leaf changes the signature and is not covered by Theorem 1.1.

1.4 Examples

For $\Gamma = \{x \le y\}$, we have $\Gamma \vDash x \times \bot \le y \times \bot$ by covariance. The reverse query fails: assign $x = \bot$ and $y = \bot \times \bot$. The left product in the reverse query has product addresses $\{\epsilon, 0\}$, while the right product has only $\{\epsilon\}$. Address 0 separates them.

Infinite models are essential to the stated domain. Let $Z = \{0^j : j \ge 0\}$. Then $Z = Z \times \bot$ in address-set notation. Thus $\{x \times \bot \le x\}$ is satisfiable, and does not entail $x \le \bot$. It has no finite-tree model: its premise forces the root of x, then its left child, then every address 0^j. Finite-tree entailment would make every query true for this input. This is a concrete reason not to replace arbitrary trees by finite trees in Theorem 1.1.

2. Expression normalization

Give each compound expression a fresh name, include the compound expressions in the query as well as those in the premises, and use a name b for bottom when it occurs. For a product name z representing $u \times v$, impose both

$$z \le u \times v, \qquad u \times v \le z.$$

Impose $b \le \bot$. Original variables retain their identities; named premises become inequalities between names. Distinct occurrences may receive distinct names. Their defining constraints give each the required interpretation; sharing repeated subexpressions is unnecessary for correctness. The four resulting clause forms are

$$x \le y, \qquad x \le y \times z, \qquad y \times z \le x, \qquad x \le \bot.$$

Lemma 2.1 (exact naming). Every original model extends to a model of the normalized constraints by assigning each new name the value of its expression. Conversely, restricting

any normalized model to the original variables gives an original model, and every expression name has its expression's value. The original and normalized entailment queries are therefore equivalent.

Proof. The extension satisfies each pair of defining inequalities by equality of interpretations. In the other direction, $b \subseteq \emptyset$ forces $b = \emptyset$. The two product inequalities force the product name to equal the product of the two component names, by antisymmetry of set inclusion. Induction over finite expression syntax gives the interpretation of every name. Apply this equality to each premise and to both query expressions. Nothing in this induction unfolds a possibly infinite value. It is an induction on input syntax only. □

We henceforth fix a normalized constraint list C on n controls and a variable query $\alpha \leq \beta$. This passage from constraints to an address automaton follows the constraint-automaton perspective of Henglein and Rehof [2, sec. 3] and Niehren and Priesnitz [3]. We give the exact top-free propagation and summary construction needed here.

3. Canonical propagation and finite stack summaries

3.1 The least model generated by seeds

A fact (x, u) means that u is a product address of variable x. The constraints induce these rules, for every word u:

Clause	Propagation
$x \leq y$	$(x, u) \to (y, u)$
$x \leq y \times z$	$(x, 0u) \to (y, u)$ and $(x, 1u) \to (z, u)$
$y \times z \leq x$	$(y, u) \to (x, 0u)$ and $(z, u) \to (x, 1u)$; also force (x, ϵ)
$x \leq \bot$	forbid every fact (x, u)

Let F_0 be the controls whose root is forced by a lower-product clause, and let N_0 be the denied controls. For a seed family S, let $\mathrm{cl}(S)$ be its closure under the five directed propagation rules in the table. Use all forced roots as seeds. To ask whether an address w separates α from β, also seed (α, u) for **every prefix** u of w. Call this finite seed set $S_{\alpha,w}$.

Lemma 3.1 (canonical separation). The query fails exactly when some finite word w satisfies

$$\mathrm{cl}(S_{\alpha,w}) \cap (N_0 \times \Sigma^*) = \emptyset, \qquad (\beta, w) \notin \mathrm{cl}(S_{\alpha,w}). \tag{3.1}$$

Proof. Each propagation rule preserves truth in every model containing its seeds. Induction on a derivation proves that the closure is contained in every such model. The closure is prefix-closed: truncate a derivation at an arbitrary address length. For an epsilon step, truncate its predecessor to the same length. For a pop producing a prefix of length i, truncate its predecessor to length $i + 1$ and perform that pop. For a push producing a nonempty prefix of length $i + 1$, truncate its predecessor to length i and perform the push. A push truncated to length zero is justified by the forced root of its target. Seed truncation is valid because the seed family is prefix-closed. Thus the closure defines trees. If no denial is reached, these trees satisfy all four clause forms, directly by the rules, including the forced root in a lower clause. They contain w at α and omit it at β exactly as in (3.1). Conversely, any failure of set inclusion has a finite separating address w. Prefix closure puts all its prefix seeds in the countermodel. Leastness then proves both conditions of (3.1). No finite-model assumption enters. □

3.2 Effective pushdown reachability

Ignore the forced roots and denials temporarily, keeping the epsilon, push, and pop rules as a finite system W on configurations (x, u), with the stack's top on the **left**. Reverse every rule to obtain W^{op}; push and pop are interchanged, and epsilon edges reverse direction.

Let $E_W(p, q)$ mean a balanced excursion: the stack ends unchanged and is never read below its original contents. It is the least relation containing identities and epsilon edges and closed under composition and

$$p \ \mathrm{push}(a)\ r, \quad E_W(r, s), \quad s \ \mathrm{pop}(a)\ q \quad \Longrightarrow \quad E_W(p, q).$$

This relation is effectively computable by saturation on n^2 pairs. Start with identities and epsilon edges, repeatedly insert every pair required by composition or a matched push/pop, and stop when no pair changes. Every inserted pair is a balanced excursion by induction. Conversely, the stable relation contains the generators and is closed under both rules; induction on a balanced derivation proves that it contains every balanced excursion. There are at most n^2 strict insertions. This is the finite summary form of pushdown reachability; its factorization is proved below.

Define $\mathrm{Pop}_W(p, u, q)$ recursively: for $u = \epsilon$, use $E_W(p, q)$; for $u = av$, take a balanced excursion, pop a, then use Pop on v. It means that the prefix u can be removed, leaving any unchanged stack suffix. It is decidable from the finite balanced table.

Lemma 3.2 (minimum-stack factorization). A configuration (p, u) reaches (q, v) if and only if there are words a, b, z and a control r such that

$$u = az, \quad v = bz, \quad \mathrm{Pop}_W(p, a, r), \quad \mathrm{Pop}_{W^{\mathrm{op}}}(q, b, r). \tag{3.2}$$

Proof. In a finite run, choose a configuration at minimum stack height. Its stack z is an unchanged suffix of both endpoint stacks. Before that configuration, each first descent past one stack letter is a pop, preceded by a balanced excursion. This gives the first Pop derivation. Reverse the remaining run to obtain the second. Conversely, a Pop derivation is valid above any suffix z. Follow the first and then the reverse of the second. The empty run, empty popped prefixes and intervening balanced excursions are included. This argument also proves the finite recursive Pop test correct. □

Existence of *some* word u with $\mathrm{Pop}_W(p, u, q)$ is finite control reachability using balanced excursions and arbitrary labelled pops. Hence we can decide whether a finite seed set reaches any denied control at any stack height. Applying this to the forced roots decides satisfiability of C. This test exhausts finite relations, not a bounded portion of an infinite stack.

3.3 One finite flagged automaton for all separating words

For a prefix u define

$$B(u) = \{r : \mathrm{Pop}_W(\alpha, u, r)\}, \qquad A(u) = \{r : \mathrm{Pop}_{W^{\mathrm{op}}}(\beta, u, r)\}.$$

Use all pairs (B, A) of control subsets as physical states: there are 4^n. The initial state is $(B(\epsilon), A(\epsilon))$. On a bit a, replace either subset S by

$$\mathrm{next}_W(S, a) = \{q : \exists p \in S, r, p \ \mathrm{pop}(a)\ r \ \wedge E_W(r, q)\}, \tag{3.3}$$

using W^{op} for the second component. The last-letter decomposition of Pop proves that the reachable state after u is exactly $(B(u), A(u))$. This interpretation is asserted for reachable states; arbitrary enumerated subsets need not already be balanced-closed.

Define the fixed danger set

$$D_0 = \{r : \exists d \in N_0, v, \mathrm{Pop}_{W^{\mathrm{op}}}(d, v, r)\}.$$

Its membership is decidable by the finite control test above. The automaton's annotations are:

- (B, A) is final iff $A \cap F_0 = \emptyset$;
- it is Bad iff C is unsatisfiable or $B \cap D_0 \neq \emptyset$;
- the ordered pair $((B, A), (B', A'))$ is flagged iff $B \cap A' \neq \emptyset$.

More generally, a **flagged instance** is a tuple $I = (Q, \delta, s, F, \mathrm{Bad}, G)$, where Q is a finite nonempty set, $\delta : Q \times \Sigma \rightharpoonup Q$ is a partial deterministic transition function, $s \in Q$, $F, \mathrm{Bad} \subseteq Q$, and $G \subseteq Q \times Q$. The preceding construction has total transitions; partial transitions will arise when unsafe edges are deleted. Call w a **witness** when its whole run $c_0, \ldots, c_N$ exists, avoids Bad, ends in F, and

$$\forall\, 0 \le i \le j \le N : \quad w[j:] \text{ is a prefix of } w[i:] \implies (c_i, c_j) \notin G. \tag{3.4}$$

The diagonal $i = j$, the empty suffix $j = N$, and the initial cut $i = 0$ are all part of this definition. In particular, a diagonally flagged state is unsafe.

Lemma 3.3 (exact summary bridge). In the constructed instance, w is a witness iff it satisfies (3.1). Consequently, $C \not\vdash \alpha \le \beta$ iff the constructed instance has a witness.

Proof. A forced root reaches (β, w) exactly when $A(w)$ meets F_0, by reversing its run. For a prefix seed (α, u), apply (3.2) to a run ending at (β, w). Write the popped-prefix lengths as i, j. Since u is a prefix of w and their remaining suffixes agree, $i \le j$ and $w[j:]$ is a prefix of $w[i:]$. The common minimum-stack control lies in $B(w[:i]) \cap A(w[:j])$. Conversely, from these data choose the seed prefix $w[:i]w[j:]$ and apply (3.2); the agreement makes it a prefix of w. This proves the flagged-pair condition, including its boundaries. A prefix seed reaches some denied control iff a popped prefix reaches a member of D_0. Thus safety of every prefix is exactly the Bad check, together with the forced-root satisfiability check. These are all canonical sources. Apply Lemma 3.1. □

4. Absorption and the terminal/cyclic decomposition

Say that an instance is **absorbing** when every defined successor q of any final state is final and has an empty incoming flag column:

$$\delta(f, a) = q,\ f \in F \quad \implies \quad q \in F \ \wedge\ \forall p \in Q, (p, q) \notin G. \tag{4.1}$$

Lemma 4.1 (summary absorption). The instance of Section 3 is absorbing, on all its physical states.

Proof. Every source of a pop in W^{op} is a target of a push in W, and hence belongs to F_0. A final state's A avoids F_0, so (3.3) makes the next A empty. Such a successor remains final; its incoming flags are empty because every intersection with its A is empty. No reachable-state interpretation of an arbitrary subset is needed here. □

A state is *safe* if it is neither Bad nor diagonally flagged. Delete edges incident with unsafe states. In this safe graph, let I_{term} keep the original state set, initial state, Bad set and flags, and make final exactly the safe **original** finals that have no remaining outgoing edge. These finals are terminal. A new sink that was not originally final stays rejecting.

Lemma 4.2 (terminal/cyclic split). For an absorbing instance, a witness exists iff either I_{term} has a witness, or an original witness has a safe continuation reaching a nonempty safe cycle.

Proof. Appending a safe path to a witness preserves witnesshood. For a flagged comparison whose later cut lies in the old word, an agreement after extension would imply its forbidden old agreement by truncation. Every newer target has an empty incoming flag column by (4.1). Finality and safety also persist. Starting at a witness endpoint, grow a safe path. If it stops, it stops at a safe original final sink. If it reaches length $|Q|$, repeated cut states supply a safe stem and a nonempty cycle. In the first case the extended word is a witness of I_{term}; in the second it has the stated cyclic continuation. Conversely, every terminalized witness lifts to the original instance, and the cyclic case already contains an original witness. The two cases need not be disjoint. □

Sections 5–9 decide terminal instances. Section 10 decides the cyclic case.

5. Reversal, border rank, and first-return milestones

Fix an instance I with k states and terminal finals. Reverse a nonempty witness w to R, and put $r_j = c_{|w|-j}$. More generally a **safe labelled reference** consists of a nonempty word $B = b_0 \cdots b_{m-1}$ and labels $r_0, \dots, r_m$ with

$$r_0 \in F, \qquad \delta(r_{j+1}, b_j) = r_j,$$

all labels outside Bad, and the reversed form of all comparisons in (3.4). Its last label need not be the original initial state s.

A border of a word is a prefix equal to a suffix; include the empty and full borders. Define the incoming column $\mathrm{col}(q) = \{p : (p, q) \in G\}$ and, at reference position j,

$$M_j = \bigcup_{b \text{ a border length of } B[:j]} \mathrm{col}(r_b). \tag{5.1}$$

Safety is exactly $r_j \notin M_j$ at every position, as well as avoidance of Bad. The full border checks the diagonal; the empty border checks flags into r_0. Indeed the comparison at original cuts $i \le j$ becomes the border of length $|w| - j$ of the reversed prefix of length $|w| - i$, with flag $(r_{|w|-i}, r_{|w|-j})$. Positive prefixes of a safe reference are safe. A safe reference ending at s reverses to an original witness.

For $j > 0$, let $\pi(j)$ be the longest proper border length of $B[: j]$, and put $\pi(0) = 0$. Every proper border is a border of this longest proper border:

$$M_j = \mathrm{col}(r_j) \cup M_{\pi(j)}. \tag{5.2}$$

Let $d(j)$ be the number of positive borders of $B[: j]$, including the whole prefix. Thus $d(0) = 0$ and $d(j) = 1 + d(\pi(j))$ for $j > 0$.

Lemma 5.1 (terminal rank bound). Every prefix of a reversed terminal accepting run has $d(j) \le k-1$.

Proof. Accepted words from a fixed state form a prefix-free language: a proper accepted prefix would leave the run at a terminal final with more letters to read. Fix an accepted suffix x of length ℓ. Map each border length b, including zero and ℓ, to the state at cut $\ell - b$ in its accepting run. From this state the accepted remaining word is $x[\ell - b :] = x[: b]$. If two distinct lengths $b < c$ had the same state, that state would accept the two prefix-comparable words $x[: b]$ and $x[: c]$, contradicting prefix-freeness. There are therefore at most k border lengths. The zero border maps to the terminal final and contributes one of them. Reversal preserves border lengths; a prefix of the reversed run is the reversal of such an accepted suffix. Removing the zero border leaves at most $k-1$ positive borders. □

This bounds a nesting rank, not word length, the number of source visits, or the number of runs of equal symbols.

For a word R, apply d and π to its prefixes: $\pi(j)$ is the parent length of $R[: j]$. Let $m_h \le |R|$ be the first position with $d(m_h) \ge h$, when it exists. The first such position has rank exactly h, since its proper-border parent is an earlier position and $d = 1 + d \circ \pi$. In particular $m_1 = 1$.

Lemma 5.2 (milestones are first returns). If m_h, m_{h+1} exist and $h > 0$, then $\pi(m_{h+1}) = m_h$, and the occurrence of $R[: m_h]$ ending at m_{h+1} is its first occurrence ending strictly after m_h.

Proof. Write $n = m_{h+1}$ and $p = \pi(n)$. Then $d(p) = h$, so $m_h \le p$. If $p > m_h$, truncate the final occurrence of the length-p border at length m_h. This makes m_h a proper border at position $n - p + m_h$, strictly between m_h and n. Its rank is at least $h+1$, contradicting firstness of n. Hence $p = m_h$. Any earlier return of that prefix would likewise have a proper border of rank h and thus rank at least $h+1$. Conversely the border at n supplies the return. Overlap is allowed throughout. □

Choose the **maximum rank over all prefixes** of R, and its first milestone. Lemma 5.2 gives at most $k-2$ first-return extensions after the first letter. The remaining suffix makes no further complete return of the last reference: a return, including one at the ultimate endpoint, would have larger rank. The complete word's rank alone is insufficient: rank falls from 3 at `000` to 1 at `0001`.

6. Prefix graphs and the exact first-return extension

6.1 A finite coloured graph for a concrete reference

The prefix graph $P(B)$ has vertices $0, \dots, m$ and four directed edge labels:

- $\mathsf{F} : j \mapsto \pi(j)$, with $\mathsf{F}(0) = 0$;
- $\mathsf{S} : j \mapsto j+1$ for $j < m$, absent at m;
- $\mathsf{K}_a : j \mapsto \kappa_B(j, a)$, for $a \in \{0, 1\}$, where κ is the length of the longest prefix of B that is a suffix of $B[: j]a$.

At $j = m$ the matching update uses the full-pattern fallback $\pi(m)$ before consuming the next letter. This is the prefix-matching update of Knuth, Morris and Pratt [6], here specified by literal prefix/suffix equality. Each vertex has colour

$$(r_j, M_j, \mathrm{next}_j, \mathrm{root}_j, \mathrm{end}_j).$$

The next component is b_j for $j < m$ and END for $j = m$. Define $\mathrm{root}_j = [j = 0]$ and $\mathrm{end}_j = [j = m]$, with Boolean values. Every fresh vertex in an extension has root false. Colours lie in

the finite alphabet $Q \times 2^Q \times \{0, 1, \mathrm{END}\} \times \{0, 1\}^2$. A numerical position is **not** part of a colour. No inverse edge is supplied.

6.2 First-return generation

Fix a safe labelled reference B. A **safe first-return extension** preserves its word and all labels through position m, appends at least one letter, and has its first new complete occurrence of B ending at the new endpoint. All old and fresh positions satisfy the reversed transition, final-root, Bad and border-mask safety conditions of Section 5. Old-label agreement is part of this definition; choosing a new label for an old position is forbidden.

Let $C = Ba_1 \cdots a_L$, $L \geq 1$, be a first-return extension, and label its fresh positions $v_i = m + i$. Define old-graph cursors

$$h_0 = \pi(m), \qquad h_i = \kappa_B(h_{i-1}, a_i), \qquad h_i \neq m \ (i < L), \quad h_L = m. \tag{6.1}$$

Choose reverse-run states $q_0 = r_m, q_1, \ldots, q_L$, where

$$\delta(q_i, a_i) = q_{i-1}, \quad q_i \notin \mathrm{Bad}, \quad (q_i, q_i) \notin G, \quad q_i \notin M_{h_i}. \tag{6.2}$$

Lemma 6.1 (generator and graph surgery). Equations (6.1)–(6.2) generate exactly the safe first-return extensions of B. Their prefix graphs are obtained by the following finite local rules.

At every old vertex $j < m$, edges and colours stay unchanged. At m, change next to a_1 and end to false, add $\mathsf{S} : m \to v_1$, and replace its K_{a_1} edge by $m \to v_1$. Its other K edge and F edge stay unchanged.

For $i < L$ the fresh colour is

$$(q_i, \mathrm{col}(q_i) \cup M_{h_i}, a_{i+1}, \mathsf{false}, \mathsf{false}),$$

and the edges are

$$\mathsf{F}(v_i) = h_i, \quad \mathsf{S}(v_i) = \mathsf{K}_{a_{i+1}}(v_i) = v_{i+1}, \quad \mathsf{K}_b(v_i) = \mathsf{K}_b(h_i) \quad (b \neq a_{i+1}). \tag{6.3}$$

At v_L, the colour is $(q_L, \mathrm{col}(q_L) \cup M_m, \mathrm{END}, \mathsf{false}, \mathsf{true})$, there is no S edge, and

$$\mathsf{F}(v_L) = m, \qquad \mathsf{K}_{a_1}(v_L) = v_1, \qquad \mathsf{K}_b(v_L) = \mathsf{K}_b(m) \quad (b \neq a_1). \tag{6.4}$$

Proof. Before the first new occurrence of B, a proper border of a fresh prefix has length less than m; at the first return its longest proper border has length exactly m. A longer proper border would contain an earlier return of B, as in Lemma 5.2. Hence the proper-border parent at v_i is exactly h_i. Equation (5.2) gives its mask and (6.2) checks precisely all new safety conditions. This proves the generator equivalence, including the final return. For matching edges, a matching next letter advances along the fresh strip. A mismatching letter falls back to the proper-border parent h_i and uses its old matching edge. At the new endpoint the proper-border parent is m; its matching a_1 edge now goes to v_1. This yields (6.3)–(6.4). Old proper prefixes cannot refer to new longer prefixes, giving the old-core rules. □

Thus every edge from the old core into the fresh strip enters at **one vertex**, v_1. Inside the strip, all edges are forward adjacent, except for the endpoint wrap $v_L \to v_1$. All other fresh edges return to the old core. For $L = 1$, the wrap is a self-loop; it must not be discarded.

6.3 Example: an overlapping return

Take $B = 0$ and $C = 00$. The occurrence ending at position 2 overlaps the old one, so $L = 1$, $h_0 = 0$, and $h_1 = 1$. There is one fresh vertex 2. Its failure edge goes to 1, its K_0 edge is a self-loop at 2, and its K_1 edge goes to 0. The old endpoint 1 now has both S and K_0 edges to 2. These statements follow directly by listing the suffixes of `00`, `000`, and `001`.

For a labelled safe example, use three states f, p, s with $\delta(p, 0) = f$, $\delta(s, 0) = p$, terminal final f, no Bad states and no flags. The original word `00` runs s, p, f. Its reversed labels are f, p, s; its two milestones are 1 and 2. The finishing continuation after the single promotion is empty. This realizes the $k - 2 = 1$ boundary exactly.

7. A finite preimage construction for directed path queries

This section is the central elimination argument. It must operate on a whole conjunction at once: choosing different generated words for different atoms would prove a weaker and unusable statement.

7.1 The query language

A path observer has a finite control set S, sets of initial and final controls, a Boolean predicate $\mathrm{allow}(s, c)$ on control and colour, and a finite transition relation

$$(s, c, \ell, d, t), \qquad \ell \in \{\mathsf{F}, \mathsf{S}, \mathsf{K}_0, \mathsf{K}_1\}.$$

Here $s, t \in S$ and c, d are colours. The tuple records an allowed observer move from control s and source colour c to control t and target colour d along an edge with label ℓ.

A graph path is accepted when its vertex/control pairs satisfy allow, consecutive pairs follow an edge and an observer transition, and its endpoint controls are initial/final. Empty paths are allowed when one control meets both endpoint conditions and allows the vertex colour. No hidden word or vertex register is part of an observer. All predicates and transition relations are represented by truth tables on the indicated finite sets.

A clause existentially quantifies finitely many vertices and conjoins finite unary colour tests, vertex equalities/disequalities, and observer path atoms $\mathrm{Path}_O(x, y)$. A query is a finite union of clauses. Different path atoms have independent existential path witnesses, except that they share the vertices named by clause variables. The language contains neither inverse edges, negated reachability, equality of independent path labels, nor set quantification.

On a finite graph, satisfaction is decidable: enumerate vertex assignments; check the finite tests and equalities; and check each path atom by reachability in the product of the graph with its observer. At most the product's number of vertices is needed for reachability saturation. A query therefore gives a concrete finite decision problem even when its observers are very large.

Theorem 7.1 (semantic promotion preimage). From a finite instance I and query φ, effectively construct a finite query $\mathrm{Pre}_I\,\varphi$ in the same language, such that for every safe labelled reference B,

$$P(B) \models \mathrm{Pre}_I\,\varphi \quad \Longleftrightarrow \quad \exists C\,[\ C \text{ is a safe first-return extension of } B\ \wedge\ P(C) \models \varphi\ ]. \tag{7.1}$$

The compiler takes **only** I and φ. It does not take B, its length, a stack-height bound, or a witness oracle. Terminality is not needed for this local theorem; it supplies the global rank bound later.

The proof has two separate directions: extract a diagram from an actual satisfying extension (coverage, Section 7.7), and reconstruct one extension from a satisfied compiled clause (Section 7.8). Both use the concrete finite envelope below. The construction below specifies the finite data, consistency tests and observers needed for both implications; no bound on reference-word length is used.

7.2 Finite diagram enumeration

Fix a clause with v variables and a path atoms whose observer sizes are $s_1, \dots, s_a$. Put $\sigma = \sum_i s_i$. For each satisfied atom in $P(C)$, choose an accepting path with no repeated vertex/control pair. Delete any cycle between repetitions to obtain such a path. This preserves the atom's endpoints and tests; its interior is not separately quantified by the clause.

For this product-simple representative, the unique fresh entrance v_1 has at most s_i visits. This is not a bound on arbitrary accepting paths. Every fresh excursion after the first must pass through v_1, either by an entrance from the core or by a wrap. The path may also start inside the strip. It therefore has at most $s_i + 1$ fresh forward blocks and at most $s_i + 2$ old-core blocks. The total block count over all atoms is at most

$$b = 3a + 2\sigma. \tag{7.2}$$

Mark the first and last fresh vertices, all fresh query-variable positions, and both ends of each fresh block. There are at most

$$g = 2 + v + 2a + 2\sigma \tag{7.3}$$

distinct marked positions. Order them by their physical position and merge coincident positions. Each block crossing a global mark records its observer control there. This includes marks introduced by *other* atoms: their positions are global, even when their controls are atom-specific.

A diagram records these mark groups; the old/fresh location of each query variable; the original-state and next-bit guesses at each group; the first emitted bit; the old endpoint colour; and a finite sequence of core, forward, entrance, exit, and wrap pieces for each atom. It also records the observer controls at piece boundaries and at crossed mark seams. Zero-length core or forward pieces are permitted; the edge pieces still denote one edge.

The following reference-independent bounds give an effective enumeration:

$$\begin{gathered} r = v + 2ab, \\ z = r + g + 2b + a + \sigma + 1; \\ \#\text{groups} \le g, \quad \#\text{auxiliary core slots} \le r, \\ \#\text{pieces in each atom} \le 2b, \quad \#\text{seam entries} \le bg. \end{gathered} \tag{7.4}$$

Here r bounds only auxiliary old-core slots. The compiled clause's total variable carrier has size at most $2 + \#\text{groups} + r$: two fixed variables e, f, one origin variable per group, and at most r auxiliary core slots. These variables are allowed to denote the same base vertex when no disequality forbids it.

Each integer field in a piece or seam is less than z. There must be at least one fresh group, since a first return is nonempty. Enumerate every datum in these ranges, every state/next-bit/colour choice, and every old/fresh location assignment. Retain exactly the diagrams satisfying these finite tests:

1. A clause variable is assigned one core slot or one fresh group. State and next-value lists have one entry per group. Only the last group has next value END. The guessed old endpoint is nonroot, has its end bit set, and has next value END.
2. Each path atom has a nonempty list of pieces. Its first and last locations equal the locations of that atom's variables, and its first and last controls belong to the observer's initial and final sets. Adjacent pieces have the same location and control at their common boundary.
3. A core piece joins two core slots. A forward piece goes from a group to an equal or later group; if the groups are equal, its two controls are equal. Otherwise, it records a valid observer control at every intermediate group. Every used seam control is in that observer's state set.
4. An entrance goes from a core slot to the first group and has label S or K_{a_1}. A wrap goes from the last group to the first with label K_{a_1}. An exit goes from a fresh group to a core slot and is either F or a mismatching K_b; the mismatch is tested against the group's next bit, or against a_1 at the last group.
5. The first fresh state obeys its reverse transition from the guessed old endpoint and is neither Bad nor diagonally flagged. The query's equalities and disequalities respect the old/fresh partition and fresh-group identity. Equalities between core slots are retained as constraints, rather than decided from the names of those slots.

Every piece has a source, a target and two observer controls. A portal piece also has one edge label. An empty accepted path is represented by a single zero-length core or forward piece. Repeated seam entries may be suppressed; one control per used atom/piece/group triple suffices. All remaining colour and edge requirements are emitted as tests in the compiled clause, as follows.

Here is why these bounds suffice. The block argument gives (7.2). Splitting at core/fresh boundaries adds at most one connecting portal edge per block, so $2b$ pieces per atom is safe. At most two old endpoints per block per atom, in addition to the v old variable slots, fit within r. Equations (7.2)–(7.3) bound crossed seams by bg. The scalar bound dominates every endpoint index, piece index, atom index, and observer-control index. This gives a diagram for any satisfying extension. The enumeration also includes redundant diagrams. It does not claim that all emitted diagrams retain the original product-simple normal form, or a small bound on the number of active observer coordinates. The proof needs finite coverage, rather than minimal diagrams.

7.3 The old-core preimage

Fix a diagram's first bit a_1 and old endpoint colour. In $P(C)$ the old-core induced graph is $P(B)$ with the old endpoint recoloured and its outgoing K_{a_1} edge suppressed. Define the finite colour map

$$\mathrm{core}_{a_1}(c) = \begin{cases} (c.\mathrm{state}, c.\mathrm{mask}, a_1, c.\mathrm{root}, \mathsf{false}) & c.\mathrm{end}, \\ c & \text{otherwise.} \end{cases}$$

For a core piece, use the same observer controls with fixed entering/leaving controls. Apply this map to both colours of each transition and to its unary allow predicates. Reject K_{a_1} when its source has the base end bit. This finite observer recognizes exactly that piece in the induced old core, however long the piece is. Old variable tests are pulled back by the same map. Old/old equalities and disequalities remain ordinary base-variable constraints.

7.4 Marks and portal edges

For every fresh group t, introduce **one** old-graph variable z_t denoting its generator origin h_i. Guess one original-state label q_t and one next value n_t for that group. They are reused by every atom touching it. Set

$$\mathrm{fresh}(q, n, c) = (q, \mathrm{col}(q) \cup c.\mathsf{mask}, n, \mathsf{false}, [n = \mathrm{END}]). \tag{7.5}$$

Require the final origin to equal the old endpoint, and all other marked origins to avoid it. Distinct physical groups **may have equal origins**; origins are matching cursors, not positions. Their physical separation is ensured by positive-length intervals below, not by $z_t \neq z_{t+1}$.

Portal pieces are local checks:

- An entrance leaves the old endpoint by S or K_{a_1} and reaches the first group.
- A wrap leaves the last group by K_{a_1} and reaches the first.
- A fresh failure exit reaches its origin z_t.
- A proper fresh mismatch on $b \neq n_t$ follows the base edge $z_t \ \mathsf{K}_b \ y$. At the final group, replace $b \neq n_t$ by $b \neq a_1$ and use $z_t = m$.

For each portal, check the original observer transition and endpoint allow predicates using the colour maps. A failure exit becomes equality with an origin and a unary test. A mismatch becomes a single-edge observer over the base graph. Entrance/wrap tests can use the guessed old-endpoint colour and the final origin, so they use only finite unary or edge predicates. No new relation between arbitrary words is introduced. Functionality of base K edges ensures that two uses of one physical mismatch edge have the same target.

Fresh/fresh equalities and disequalities are decided by group identity. Old/fresh equality is impossible and old/fresh disequality automatic. Pull back fresh variable colour tests through (7.5).

7.5 One common generator on every physical interval

Let the distinct fresh groups be $t = 0, \ldots, h-1$, from the first fresh vertex to the last. Use two base variables e, f and the one-edge constraints

$$e \ \mathsf{F} \ f, \qquad f \ \mathsf{K}_{a_1} \ z_0.$$

Require e to have the guessed old-endpoint colour. Write q_{old} for that colour's state and q_{fresh} for the state guessed at the first fresh group. The diagram checks

$$\delta(q_{\mathrm{fresh}}, a_1) = q_{\mathrm{old}}, \qquad q_{\mathrm{fresh}} \notin \mathrm{Bad}, \qquad (q_{\mathrm{fresh}}, q_{\mathrm{fresh}}) \notin G.$$

The second edge additionally checks q_{fresh} against the mask at z_0, namely M_{h_1}. Thus these states are q_0 and q_1, respectively, in the generator notation of Section 6.2. Fresh groups are numbered from zero. Its target is End exactly when $h = 1$. Thus one fresh group handles the single-emission case directly.

For two consecutive distinct groups, all forward pieces traversing that physical interval are known. Split every such piece at all intermediate marks and retain its control at each seam. If one original path crosses an interval twice, it contributes **two coordinates**, not one.

Let ℓ be the number of active forward pieces on this interval, counting repeated traversals separately. Make a product observer whose control is

$$(q, n, \eta, t_1, \dots, t_\ell), \tag{7.6}$$

where $q \in Q$ is the current generated reverse state, n is the next bit or END, $\eta \in \{0, 1\}$ records whether a step has occurred, and each t_j is the current control of one active forward piece. Its size is $6k \prod_{j=1}^{\ell} |S_j|$. Even when there are no active pieces, retain the generator coordinates; that unobserved interval still has to exist and be safe.

At a base origin of colour c, allow a control only if

$$q \notin \mathrm{Bad}, \quad (q, q) \notin G, \quad q \notin c.\mathrm{mask}, \quad (n = \mathrm{END}) \iff c.\mathrm{end},$$

and every active observer allows the single produced colour $\mathrm{fresh}(q, n, c)$. A step can read only a base K_a edge $c \to d$, with $n = a$ and c not End. Choose q' with $\delta(q', a) = q$, check its safety and the mask of d, and choose the next value n' (END exactly at an End origin). Each active coordinate independently chooses an S or K_a transition, but **all coordinates see the same** a **and the same two produced colours**. Set $\eta' = 1$.

Initial and final tuples are fixed by the shared boundary guesses of the diagram, with initial $\eta = 0$ and final $\eta = 1$. This enforces positive length between distinct groups. END has no outgoing generator step. Therefore an earlier full return cannot be hidden in an interval. The next-bit lookahead is a finite guess verified by the next step, shared at adjacent seams.

One regular-path atom between z_t and z_{t+1} now says that one common generator segment realizes every active observation on this physical interval. Concatenate these atoms through their shared endpoint data. Together with the first emission they specify exactly one first-return generation, rather than one generation per original atom. Core path witnesses remain independent, as the original query semantics allow.

7.6 Assembly of the finite query

Conjoin the pulled-back tests, equalities, portal constraints, core-path atoms, first-emission constraints, and one product-path atom for each interval. Existentially quantify their base vertices. This is a clause of the original query language. Disjoin over the finite diagrams and over the clauses of φ to obtain $\mathrm{Pre}_I \varphi$.

Every loop in this construction is over a finite syntax bound, colour alphabet, state set, or observer product. The compiler never enumerates reference words and never needs to bound $|B|$ or $|C|$.

7.7 Proof of coverage: a promoted model gives a base model

Suppose a safe first-return extension C satisfies a clause. Fix its variable assignment and accepting product-simple paths. Section 7.2 provides a diagram inside the enumerated envelope. Interpret its fresh groups at their actual positions in the one strip of C, and its origin variables at the actual cursors h_i. Use their actual states, next bits, endpoint colour and observer controls for all guesses. The old/fresh partition preserves equality, including coincident roles. Lemma 6.1 gives every portal edge and old-core rewrite exactly.

Split the actual generator at the global marks. In each interval, pair this same generator with each active piece's actual observer run. Their product satisfies (7.6), since these observations already traverse those same physical vertices in C. Positivity follows from distinct ordered marks. Hence the compiled base clause is satisfied. The satisfying diagram is enumerated by (7.4); no completeness assumption about a heuristic subset of diagrams is used.

7.8 Proof of reconstruction: a base model gives one promoted model

Conversely, suppose one compiled base clause is satisfied. Choose witnessing paths for its interval atoms. Their shared origins, reverse states, next values and seam controls match. Together with the first-emission edge, concatenate them to form **one** generator walk. Positivity puts consecutive groups at distinct ordered positions; the End guards force the last emission to be its first complete return. Equations (6.1)–(6.2) therefore give a definite safe reference C, and Lemma 6.1 gives its exact graph.

Place every fresh query variable at its shared physical group in this strip, and each old variable at its base assignment. The colour tests and equality partition are correct by construction. Product coordinates supply forward observer paths on each interval of this same strip. Join them across seams for each forward piece. Join these pieces with the core paths and portal edges prescribed by the diagram. The stored controls agree at every join, so each original observer accepts its reconstructed path. Zero-length pieces use their endpoint allow/control tests; a single-group wrap uses Lemma 6.1's genuine self-loop. Thus the original clause holds in $P(C)$, proving (7.1). □

7.9 Example: two simultaneous observations

Take four original states f, p, q, s, initial state s, final state f, no Bad states or flags, and only the transitions

$$\delta(p,0) = f, \qquad \delta(q,1) = p, \qquad \delta(s,0) = q.$$

Use the safe reference $B = 0$ with labels (f, p). Its first-return extension $C = 010$ has reversed labels (f, p, q, s). The fresh positions are $v_1 = 2$ and $v_2 = 3$; their old-graph origins are $h_1 = 0$ and $h_2 = 1$. All masks are empty.

The query has two variables x, y, tests that their state colours are q, s, and two path atoms from x to y. Each observer has controls $\{0, 1\}$, initial control 0, final control 1 and unrestricted colour allowance. The first has the single move $0 \xrightarrow{\mathsf{S}} 1$; the second has the single move $0 \xrightarrow{\mathsf{K}_0} 1$. Both hold at $x = v_1, y = v_2$.

Here is every nontrivial guess of a satisfying diagram. There are two fresh groups, for v_1, v_2; x belongs to group 0 and y to group 1. Their state guesses are (q, s), and their next guesses are $(0, \mathrm{END})$. The first emitted bit is 1. The guessed old-end colour is $(p, \emptyset, \mathrm{END}, \mathsf{false}, \mathsf{true})$. Each atom has one forward piece from group 0 to group 1, with observer controls 0 and 1. There are no internal seams, core pieces or portal pieces for these two paths. There are no equality or disequality constraints.

Assign the base variables $e = 1$, $f_0 = 0$, $z_0 = 0$, $z_1 = 1$. The first-emission conditions are $e \mathsf{F} f_0$ and $f_0 \mathsf{K}_1 z_0$, since $\pi(1) = 0$ and $\kappa_B(0, 1) = 0$; the reverse-state check is $\delta(q, 1) = p$. There is one physical interval. Its product observer goes from $(q, 0, 0, 0, 0)$ to $(s, \mathrm{END}, 1, 1, 1)$ over the single base edge $z_0 \mathsf{K}_0 z_1$. The last two coordinates belong to the two different observers; the third records that a step occurred. The common emitted bit is 0, $\delta(s, 0) = q$, and both observations permit their matching forward edge. The endpoint is the old end, as required by END.

Reconstruction emits first 1 and then 0, obtaining exactly $C = 010$ and both paths in that one graph. Replacing the second observer's sole label by K_1 would remove this product transition: the first observer and the second cannot choose different emissions for the same physical interval.

7.10 Shared-word consistency

Suppose two forward observations cover the same interval, one requiring the first emitted bit to be 0, the other requiring it to be 1. Each may have a separately satisfying generated extension. Their conjunction has none. An independent preimage per atom would incorrectly accept; the single product in (7.6) cannot make both transitions on its common first bit. The same issue arises with incompatible lengths, repeated traversals, and marks contributed by another atom. Sharing only endpoints would not repair it.

7.11 The directedness restriction

For $B = 0$ and $C = 0\,1^n\,0$, every internal prefix ending in 1 has failure parent 0. If inverse failure edges were available, the old root would have edges to arbitrarily many fresh vertices. The one-entrance proof would fail. Our compiler introduces no inverse edge. Nor does it introduce equality of independent path labels or negative reachability. Theorem 7.1 asserts closure only for the specified positive directed query language.

8. Completion without another return

Starting at $h_0 = \pi(m)$ in $P(B)$ with original control r_m, follow only K_a edges. On such an edge from cursor h to h', choose a predecessor control p with

$$\delta(p, a) = q, \quad p \notin \mathrm{Bad}, \quad (p, p) \notin G, \quad p \notin M_{h'}, \quad h' \neq m.$$

The observer also excludes m at its starting cursor and all later vertices; $\pi(m) < m$ because B is nonempty. Accept when the original control is s. The empty walk is allowed when $r_m = s$. This is a finite observer with k controls. Introduce an endpoint variable e, a variable f with one F edge from e, and a final cursor variable x. Choose the endpoint state by a finite disjunction. The resulting query is Finish_I. More explicitly, its clause for state q conjoins the test that e is the endpoint with state colour q, the one-edge atom $e\ \mathsf{F}\ f$, and a continuation-observer atom from f to x starting with control q. Thus the observer starts at the proper-border cursor with the old endpoint's original control; those are two different coordinates.

Lemma 8.1 (finishing). For a safe reference B, $P(B) \models \mathrm{Finish}_I$ iff some safe extension ending at original state s makes no further complete return of B.

Proof. Before a complete return, every new proper border is shorter than B; otherwise truncating it would give an earlier return. Thus the matching cursor lists the longest proper border, and its mask contains exactly all remaining proper-border columns. The new state's diagonal is checked separately. This is precisely the step condition above. Induction over the observer path gives a safe labelled completion, whose final state is s. Conversely any such completion determines its matching-cursor path and the required state choices. Empty completion is included. Reversing the completed reference gives an original witness by Section 5. □

9. Terminal decision by finite preimage iteration

Set

$$\Phi_0 = \mathrm{Finish}_I, \qquad \Phi_{h+1} = \Phi_h \vee \mathrm{Pre}_I\, \Phi_h. \tag{9.1}$$

Lemma 9.1 (semantic unwind). For a safe reference B, $P(B) \models \Phi_h$ iff there is a chain of at most h safe first-return extensions from B to a reference satisfying Finish_I.

Proof. Induct on h. At zero the chain is empty. At a successor, the first disjunct permits the shorter chain; Theorem 7.1 applied to the second supplies its first return and a remaining chain of at most h returns. In the reverse direction, an empty chain satisfies the inherited disjunct; otherwise apply Theorem 7.1 to its first return and the induction hypothesis to the remaining chain. Safety of every intermediate reference is part of the first-return relation. □

Theorem 9.2 (terminal decision). Witness existence is decidable for every finite binary instance with terminal finals.

Algorithm. Check the empty word directly. If $k \geq 2$, construct Φ_{k-2}. Enumerate both letters a and all k^2 pairs of labels (f, p). Retain the safe one-letter references with labels f, p, including $\delta(p, a) = f$ and all boundary checks. Evaluate Φ_{k-2} on each of their two-vertex prefix graphs. Accept iff one test succeeds. For $k = 1$, no nonempty terminal accepting word is possible: its zero and full borders would require two distinct states by Lemma 5.1. An instance with $k = 0$ has no possible initial state.

The enumerated reference has word a and labels f, p at positions 0 and 1. Its prefix graph exposes exactly those positions. Any extension of its label function beyond position 1 is irrelevant; thus this finite enumeration covers the first-letter graph of every nonempty safe reference.

Termination. Equation (9.1) performs exactly $k - 2$ finite syntactic operations of Theorem 7.1. The remaining enumeration and finite-graph query tests terminate by Section 7.1. This is a complete finite construction, not an unbounded search waiting to discover a witness or a length bound.

Soundness. By Lemma 9.1, a successful test reconstructs a finite safe chain and then, by Lemma 8.1, a completed reference ending at s. Its reversal satisfies the original witness predicate. Soundness does not use terminality.

Completeness. Reverse any nonempty original witness. Choose its maximum rank over **all** prefixes and its first milestone. Lemma 5.1 bounds that rank by $k-1$. Lemma 5.2 expresses the reference up to that milestone as a chain of at most $k - 2$ first returns from its first letter. Its remaining completion has no further return, including at the final cut, so Lemma 8.1 makes the finishing query true there. Lemma 9.1 makes Φ_{k-2} true on the initial one-letter graph, which the algorithm enumerates. The empty witness was checked separately. □

10. Decision of infinite continuation

Now let I be an absorbing instance with k states, without assuming terminal finals. A witness with a safe cyclic future can be extended to a witness ending at a safe final state q with empty incoming flag column, on a safe nonempty cycle c of length $d \leq k$. To see this, first take a positive safe extension to enter the empty-column region; absorption keeps that region closed along the chosen continuation. Repeated states in a finite safe graph supply a nonempty cycle of length at most k. Lemma 4.2's safe-extension argument preserves the witness while moving to its base. The prefix can remain arbitrarily long; only the cycle is bounded.

Enumerate all states q and binary words c of lengths 1 through k. Retain exactly safe cycles from q to itself with q final and its whole incoming flag column empty. For one retained pair, ask whether a safe word v runs from s to q such that the infinite stream vc^ω has no equal suffixes at an ordered flagged pair of prefix cuts. Diagonal flags are already excluded by safety; targets strictly after v have empty columns.

10.1 The periodic suffix criterion

Write $n = |v|$, and let r be the d-periodic stream aligned with c^ω at cut n. Suppose $x = vc^\omega$ and $i < j$. If $x[i:] = x[j:]$, it has period $j - i$ from i onwards. For any $t \geq i$, add a sufficiently large multiple of $d(j - i)$ to move beyond n. Periodicity of x along $j - i$ and of r along d gives $x(t) = r(t)$. Hence

$$x[i:] = x[j:] \iff [x \text{ agrees with } r \text{ from } i \text{ onwards}] \ \wedge \ [r[i:] = r[j:]]. \tag{10.1}$$

Equality of the last two rotations is checked on their d letters. Distinct phases may give the same rotation when c is not primitive; comparing phase numbers alone would be incorrect.

10.2 A complete finite reverse monitor

Scan a prospective prefix from right to left. A monitor state consists of:

1. the current original control;
2. a phase in $\{0, \dots, d - 1\}$;
3. a Boolean saying whether the scanned suffix matches its aligned periodic ray;
4. for each phase, a set of original controls at the already scanned later cuts.

Use the full finite carrier, of size at most $2kd\,2^{kd}$. The seed is at control q, phase zero, matching true, with q in bucket zero only. Require q safe. For a reverse edge, choose a bit a and safe predecessor p with $\delta(p, a)$ equal to the current control.

While matching, decrement phase modulo d. If a is not the cycle letter at that phase, set matching false and reset the phase and buckets. If it matches, reject when p flags a stored control in any bucket whose periodic rotation equals the new phase's rotation. Otherwise add p to that bucket. Once matching is false, keep it false and keep the canonical reset data. Accept exactly the monitor states whose original control is the instance's initial state $s = I.\text{initial}$.

These are effective transitions on finite data. Undefined or unsafe original edges produce no monitor edge. Reachability from the seed to the accepting set therefore terminates by finite graph search.

Lemma 10.1 (monitor equivalence). Fix one of the retained cycles: its word c is nonempty, it is safe, and its base q is final with empty incoming flag column. An accepting reverse path from its safe seed represents exactly a safe prefix v to q with no flagged equal infinite suffixes among its prefix cuts.

Proof. While matching, the buckets contain exactly the controls of the later cuts, grouped by phase. Equation (10.1) identifies the new forbidden pairs as precisely a flag from the new source p to a stored target at an equal rotation. Sets suffice; multiplicities do not change this existence test. All old pairs were checked earlier. At the first mismatch from the right, every earlier suffix contains that mismatch and so cannot equal a later suffix by (10.1). No further buckets are needed, but transition and safety checks remain necessary. A previously rejected pair between later cuts cannot be repaired by prepending a mismatch: its two suffixes do not change. Induction proves soundness. For completeness, use the actual cuts and phases of any safe good prefix as the buckets. Every stored member then has a genuine later cut, so a rejected conflict would contradict the assumed absence of flagged equal suffixes. This builds the reverse path. The empty prefix is the seed case. □

10.3 Return from an infinite stream to a finite witness

Lemma 10.2 (finite extraction). A good prefix accepted by this monitor gives an original finite witness with a safe cyclic continuation. Conversely, a witness ending at the chosen cycle base gives a good prefix.

Proof. For a good prefix of length n, each forbidden pair of old cuts has a mismatch in their infinite suffixes. In fact a mismatch occurs before $n+d$ offsets: after n offsets both suffixes lie in the periodic tail, and d further offsets determine all later equality. Extend the prefix along the safe cycle to length at least $2n+d$. Every old target comparison now tests at least $n+d$ letters and contains its mismatch. Every newer target column is empty by absorption. The whole finite word is safe and final and still has an infinite safe future. Alternatively choose vc^{n+1}, which ends at the same cycle base and has sufficient length. Conversely, an original witness already contains a mismatch for each flagged pair; appending an infinite continuation cannot erase it. □

Theorem 10.3 (infinite-continuation decision). On a finite absorbing instance, the cycle enumeration followed by the finite monitor tests decides whether some original witness has a safe cyclic continuation.

Proof. The cycle reduction covers every such witness. Lemma 10.1–Lemma 10.2 prove both directions for each enumerated cycle. There are finitely many cycles to test, and each monitor is finite. □

11. The entailment decision procedure

Normalize the expressions by Lemma 2.1, construct the summary instance I of Lemma 3.3, and compute

$$D(\Gamma, s, t) = \neg(\mathrm{InfiniteWitnessCheck}(I) \ \vee \ \mathrm{TerminalWitnessCheck}(I_{\mathrm{term}})). \tag{11.1}$$

Lemma 4.1 supplies absorption without a caller hypothesis. Lemma 4.2 gives the exact split, Theorem 9.2 and Theorem 10.3 decide its branches, Lemma 3.3 identifies their disjunction with normalized non-entailment, and Lemma 2.1 transports it to the original expressions. This proves Theorem 1.1, including unsatisfiable inputs and arbitrary infinite valuations.

Every stage terminates on finite input: syntax traversal, finite relation and subset enumeration, a finite safe-graph transformation, bounded query promotion, finite graph satisfaction, and bounded cycle enumeration with finite monitors. The word lengths quantified in the semantic proofs are not implicit search cutoffs. Classical reasoning may select witnesses inside erased correctness proofs; the executable definitions enumerate concrete finite data and do not use classical choice as a runtime decision procedure.

11.1 A uniform computable witness bound

Corollary 11.1. There is a total computable function L such that every terminal k-state instance with a witness has one of length at most $L(k)$.

Proof. For $k \geq 1$ there are

$$k(k+1)^{2k}2^{2k+k^2}$$

labelled partial binary instances: choose the initial state, two partial successors per state, final and Bad bits, and ordered flags. Filter terminal finals and decide every remaining instance with Theorem 9.2. For each positive instance, enumerate finite words by length until its decidable

original witness predicate succeeds. This terminates on every positive instance. Take the maximum of the resulting lengths, or zero if there are none. Adding one bounds every source's number of visits. There is no instance with zero states; define the bound there as zero.

Every finite classification and every positive-instance search terminates, so their maximum is a total computable function of k. □

This corollary gives no useful running-time estimate. An explicit resource analysis must also bound the generated query syntax and its evaluation.

12. Comparison with restricted entailment results

12.1 The constant-free fragment

The absence of a constant from the input syntax does not restrict the values assigned to its variables. For example, consider

$$x = x \times x, \qquad x \leq y, \qquad \text{with query } y \leq x.$$

Over the present bottom/product domain the equation forces $x = \Omega = \Sigma^*$, and hence $y = x$. In a domain with an explicit top leaf, the assignment $x = \Omega$, $y = \top$ satisfies the premises and falsifies the query. Thus identity translation of constant-free constraints between these two domains is unsound.

There is, nevertheless, a polynomial reduction of our constant-free input fragment to the restricted problem of Niehren and Priesnitz [4, Theorem 14]. The target permits product equations and inequalities between variables, with valuations in the bottom/product/top domain. Its order is the greatest relation satisfying the following rule: $X \leq Y$ exactly when $X = \bot$, or $Y = \top$, or both roots are products and their corresponding children are ordered. There are no other comparisons of root labels. Name every product subexpression as in Section 2, writing its two defining inequalities as a single product equation. Add a fresh variable ω and the constraints

$$\omega = \omega \times \omega, \qquad z \leq \omega \quad \text{for every original or auxiliary name } z.$$

Proposition 12.1. This construction preserves entailment over arbitrary trees. Consequently, our constant-free fragment belongs to PSPACE.

Proof. Equality with its own product forces the root of ω to be a product. Repeating the equation at each finite address forces every node to be a product; hence $\omega = \Omega$. In the target order, $X \leq \Omega$ holds exactly for trees X with no top-labelled node. One direction follows by following the product children until any top label, which cannot lie below a product. The other follows directly from the pointwise covariant order with bottom least.

Every source valuation extends to a target valuation by setting ω to Ω and interpreting each auxiliary name by its expression. Conversely, the guards in a target model ensure that each named value is top-free. Restriction therefore gives a source model; the product equations preserve all expression values and the query. Both model maps preserve any failure of the query as well as its truth. Rename only the variables that occur in the input. There is at most one new name and one equation per product occurrence, and one guard per name. The construction is polynomial in the concrete input length, including variable indices; it does not enumerate an unused ambient variable carrier. The PSPACE upper bound follows from the cited restricted entailment theorem. □

This proposition concerns inputs without bottom literals. It does not remove denial constraints from the full problem of Theorem 1.1. Its equation for ω also requires an infinite tree, so the same construction gives no finite-tree transfer. The upper bound is inherited from the earlier result; no matching PSPACE lower bound for our variable domain follows from it.

12.2 Effective regular countermodels

A regular tree has only finitely many distinct subtrees, equivalently a finite graph representation with bottom nodes of outdegree zero and product nodes with a designated child for each bit. The address method also gives a constructive regular-countermodel property, compatible with the established regular-model results in the entailment literature [3; 5].

Proposition 12.2. If an input of Theorem 1.1 fails, it has an effectively constructible regular countermodel. Thus entailment over arbitrary trees is equivalent to entailment over regular trees for the same inputs.

Proof. First consider a normalized failed query. By Lemma 3.1, some finite word w gives a separating canonical closure with the finite seed set $S_{\alpha,w}$. Let Q_W be the controls of W. For $q, r \in Q_W$, define

$$L(q, r) = \{b : \mathrm{Pop}_{W^{\mathrm{op}}}(q, b, r)\}.$$

This language has a finite epsilon-NFA: its states are the controls of W, its initial state is q, its accepting state is r, its epsilon edges are all balanced pairs of W^{op}, and its a-edges are the pop-a rules of W^{op}. Epsilon steps are allowed before, between and after letters. Induction on the consumed word gives exactly the recursive Pop definition, including balanced reachability for the empty word. All its edges are computable by Section 3.2.

For each seed (p, u), Lemma 3.2 gives the exact language equality

$$\{v : (p, u) \text{ reaches } (q, v)\} = \bigcup_{\substack{0 \le i \le |u|,\ r \in Q_W \\ \mathrm{Pop}_W(p, u[:i], r)}} L(q, r)\,\{u[i:]\}.$$

The fixed suffix is on the right. The union is finite, and its terms are selected by decidable Pop tests. Appending a fixed suffix and taking a finite union preserve effective regularity. Union over the finite seeds therefore gives a finite NFA for the product-address language D_q of every control. Lemma 3.1 supplies prefix closure, denial safety and the model property.

Determinize each NFA. Create one product node for each reachable accepting DFA state, and one bottom node. From a product node, the a-child is its DFA successor when that state accepts, and bottom otherwise. Choose the initial DFA state as root if it accepts, and bottom otherwise. Prefix closure implies that a rejected word has no accepted extension. Hence induction along an address shows that this graph unfolds to a tree with product addresses exactly D_q. The resulting regular valuation is the same separating canonical model.

Products and bottom preserve finite graph representations, while restriction to original variables selects existing roots. The two naming maps of Lemma 2.1 therefore preserve regularity, giving the full-expression result. For effectiveness, first decide failure using Theorem 1.1, then enumerate finite words until the decidable test (3.1) finds a separating word; this search terminates on a failed input. Apply the finite constructions above. Finally, a regular countermodel is itself an arbitrary-tree countermodel, which proves the converse implication for entailment. □

This extraction has no efficiency guarantee. It also does not supply a finite-tree countermodel: the constructed graph may have cycles.

12.3 Scope of the contribution

The distinction from prior automata characterizations lies in the effective elimination in Theorem 7.1. The common generated word is not replaced by independent choices for individual path atoms. The one-entrance property bounds the number of pieces in product-simple paths, and the common interval observer enforces their simultaneous realizability. This gives a finite construction that can be iterated using the terminal rank bound. Standard automata reductions and word periodicity remain essential components of the argument, rather than novelty claims in isolation.

The richer first-order theories studied by Su, Aiken, Niehren, Priesnitz and Treinen [7] have different logical quantifiers and cannot be identified with the entailment fragment here. Later work of Breuvart, Penelle and Rehof [8] describes periodic selections in a cap-language setting. We do not use that slide presentation as a proof of the whole-conjunction preimage theorem established here. Schubert's proposed PSPACE procedure [9] was withdrawn following an incorrectly formulated lemma, and is not used as a decision theorem. These comparisons delimit the results on which the present proof relies; they do not assert that the general nonstructural problem with explicit top has been resolved.

13. Entailment over finite trees

Finite-tree entailment has the same syntax and subtyping order, but a smaller valuation domain. We continue to write $C \models \alpha \le \beta$ for arbitrary trees, and write $C \models_{\mathrm{fin}} \alpha \le \beta$ when every valuation of the normalized controls by finite trees that satisfies C also satisfies the query. The reduction below depends on a property of the normalized system: every push targets a control whose product root is forced by a lower-product clause. It would fail for a pushdown system with unforced pushes.

13.1 The finite growth test

Let W be the propagation system on n controls from Section 3. The reverse-pop automaton $\mathcal{A}_C$ has these controls as states and alphabet $\{0, 1\}$. Its epsilon edges are the balanced pairs of W^{op}; its a-edges are the pop-a rules of W^{op}. For initial state q and sole accepting state m, it accepts precisely

$$L(q, m) = \{w : \mathrm{Pop}_{W^{\mathrm{op}}}(q, w, m)\}.$$

Indeed, an empty accepted word is a balanced excursion; each consumed letter is a pop preceded by balanced motion, followed by the same condition on the remaining suffix. This is the recursive Pop definition of Section 3.2. The balanced relation is computable by finite saturation.

Let T be reflexive transitive reachability in this automaton after forgetting edge labels, retaining both balanced epsilon edges and letter edges. Define $\mathrm{Growth}(C)$ by the finite test

$$\exists p, r, a \quad p \xrightarrow{a} r \text{ in } \mathcal{A}_C \quad \text{and} \quad T(r, p). \tag{13.1}$$

Thus growth means a closed walk containing at least one letter edge, called a **consuming cycle**. An epsilon-only cycle does not count. Compute T by finite graph closure and inspect the finitely many letter edges. In particular, internal push/pop steps of a balanced excursion are not consumed input letters.

Lemma 13.1 (acyclic word bound). If $\mathrm{Growth}(C)$ is false, then $\mathrm{Pop}_{W^{\mathrm{op}}}(q, w, m)$ implies $|w| < n$, for all controls q, m and words w.

Proof. A nonempty accepted loop can be rotated to start at one of its letter edges. The rest of the loop is then a return path, satisfying (13.1). This remains valid when the original loop begins with balanced epsilon motion. More explicitly, decompose that loop as $E_{W^{\mathrm{op}}}(p, r)$, a pop $r \xrightarrow{a} s$, and $\mathrm{Pop}(s, v, p)$. Append the initial balanced motion to the last part; it becomes $\mathrm{Pop}(s, v, r)$, a return for the selected letter edge.

An accepting run for w has $|w| + 1$ boundary controls: its initial control and the control just after each letter edge. If two of these controls agreed, the intervening subrun would consume a nonempty subword and form such a loop. They are therefore distinct. There are only n controls, so $|w| + 1 \leq n$. Trailing epsilon steps do not affect this count. If $n = 0$, there are no initial or accepting controls and the statement is vacuous. □

Lemma 13.2 (forced growth). If $\mathrm{Growth}(C)$ holds, then C has no finite-tree model.

Proof. Choose a letter edge $p \xrightarrow{a} r$ with a return path. That edge reverses a push into p in W. By the propagation table, a lower-product clause forces (p, ϵ) in every model. Reading the return path gives a word v and $\mathrm{Pop}_{W^{\mathrm{op}}}(r, v, p)$. Consequently $\mathrm{Pop}_{W^{\mathrm{op}}}(p, av, p)$.

Put $w = av$, so $w \neq \epsilon$. Reverse this Pop run and lift it over any unchanged suffix z, obtaining

$$(p, z) \longrightarrow_W^* (p, wz).$$

Starting from the forced root and repeating the run makes w^j a product address at p for every $j \geq 0$. Propagation is sound in every C-model, and these addresses have unbounded length. The value at p cannot be finite. □

13.2 Finite canonical replacements and the decision

Suppose $\mathrm{Growth}(C)$ is false. Let S be a finite seed family whose words have length at most d. For any run $(p, u) \to_W^* (q, v)$ from a seed, Lemma 3.2 gives

$$u = az, \qquad v = bz, \qquad \mathrm{Pop}_W(p, a, r), \qquad \mathrm{Pop}_{W^{\mathrm{op}}}(q, b, r).$$

Lemma 13.1 bounds $|b| < n$, while z is a suffix of u, so $|z| \leq d$. Hence every reached word satisfies

$$|v| = |b| + |z| < n + d. \tag{13.2}$$

There are only finitely many binary words below this length and finitely many controls. The closure is therefore finite. An empty seed family has empty closure. This argument bounds the closure of the supplied seeds; it makes no assertion about the length of the shortest separating address.

Theorem 13.3 (finite-model transfer). For every normalized C and query $\alpha \leq \beta$,

$$C \vDash_{\mathrm{fin}} \alpha \leq \beta \quad \Longleftrightarrow \quad \mathrm{Growth}(C) \ \vee \ (C \vDash \alpha \leq \beta). \tag{13.3}$$

Proof. If growth holds, Lemma 13.2 makes every finite-tree query true vacuously. If arbitrary entailment holds, its restriction to finite valuations also holds.

For the converse, argue by contraposition. Suppose neither disjunct holds. A countermodel to arbitrary entailment has a finite separating address w. The finite seed set $S_{\alpha,w}$ of Lemma 3.1 lies inside that countermodel. Its canonical closure therefore avoids all denials and omits (β, w),

while containing (α, w). It is a model of C by the same lemma. Equation (13.2), with $d = |w|$, makes every component tree finite. Thus it also refutes finite-tree entailment. This proves both directions. □

To obtain Theorem 1.2, first normalize the finite expressions. The extension in Lemma 2.1 preserves finiteness: bottom has no product addresses, and a product of two finite trees adds one root to two finite tagged address sets. Structural induction over expressions proves that every auxiliary value is finite. Restriction to the original variables also preserves finiteness. Thus both model maps and both query interpretations restrict exactly to the finite domain. Compute (13.1) on the normalized system; return true if it holds, and otherwise use D from Theorem 1.1. The finite graph computations and D terminate, and (13.3) proves soundness and completeness of the result.

Corollary 13.4 (finite satisfiability). A normalized C has a finite model if and only if it has an arbitrary model and Growth(C) is false.

Proof. A finite model is an arbitrary model, and Lemma 13.2 rules out growth. For the converse, take the closure of the forced-root seeds alone. It is contained in the assumed model, hence denial-safe; the canonical-model argument of Lemma 3.1 makes it a model. Equation (13.2), now with $d = 0$, makes it finite. □

Absence of growth does not make every model finite. An unconstrained variable may still denote an infinite tree. What the proof supplies is a finite replacement for a failed query or a satisfiable constraint set.

13.3 Examples and scope

The premise $x \times \bot \leq x$ can be written in normalized form as $x \times b \leq x$ and $b \leq \bot$. A reverse-pop left-letter loop returns to the forced root at x. Lemma 13.2 explains the absence of finite models seen in Section 1. The query $x \leq \bot$ is therefore true under finite entailment and false under arbitrary entailment, witnessed by the infinite left spine.

For $C = \{x \leq y\}$, there are no letter edges and no growth. The query $y \leq x$ fails in both domains: $x = \bot$, $y = \bot \times \bot$ is a finite countermodel, separated at ϵ.

Finally, take the normalized constraints $x \times x \leq b$ and $b \leq \bot$. The only letter edges of the reverse-pop automaton go from b to x; none returns. Growth is false, but the forced root at b violates its denial. The input is unsatisfiable in both domains. A negative growth test is therefore not, by itself, a satisfiability decision.

The finite result concerns the same bottom/product order as Theorem 1.1. It does not transfer the structural finite-type coNP theorem of Henglein and Rehof, nor their nonstructural finite-tree PSPACE hardness in a domain admitting explicit top. Those statements require their own semantic hypotheses [1, Corollary 4.9; 2, Theorem 2]. Likewise, the finite-tree case of Niehren–Priesnitz's restricted entailment theorem uses the bottom/product/top valuation domain [4, Theorem 14]. The guard $\omega = \omega \times \omega$ from Proposition 12.1 has no finite solution, so that particular reduction cannot transfer their finite upper bound to the present problem. The cap-automaton characterization also distinguishes finite from infinite trees [3]; the consuming-cycle argument here is specific to the forced-target property of our top-free propagation system. No PSPACE upper or lower bound for the full finite-tree problem follows from these comparisons alone.

14. Formalization and further questions

The decision procedure is implemented in Lean, together with a proof that its Boolean result is equivalent to the semantic entailment definition for every finite input. The quantified valuations are arbitrary prefix-closed sets; neither regularity nor decidable membership is imposed on them. The executable construction enumerates finite data. Classical reasoning used inside proof terms does not provide a runtime decision oracle. The reference implementation computes some finite relations by exhaustive table enumeration; a separate saturation implementation computes the same least closed relations. Proved equivalences justify the optimized entry point.

The finite-tree Boolean has a separate correctness theorem with the restricted valuation predicate of Theorem 1.2. Its executable cycle test supplies either the forced-loop obstruction or the acyclic word bound. Neither is an assumption left to the caller. The argument then uses the same normalization and arbitrary entailment implementation.

The accompanying artifact [10] contains the source, reproduction instructions, examples and a result-by-result map with twenty-one entries: seventeen for the lemmas and theorems supporting the arbitrary-tree decision in Sections 1–10, and four for the finite-tree results in Sections 1 and 13. Corollaries 11.1 and 13.4 and Propositions 12.1–12.2 are ordinary mathematical consequences proved here; they are identified separately from the checked declarations. The explicit seed-dependent estimate (13.2) is derived in the text; the formal semantic transfer needs the existence of a uniform bound. The correspondence map also distinguishes the mathematical dependencies from the larger source import graph.

Several questions remain. The finite query construction is large, and the qualitative argument does not give a tight complexity classification or a practical worst-case algorithm. Proved finite saturation and sound early certificates improve evaluation, but do not eliminate all expensive cases. The artifact also implements the extraction in Proposition 12.2 and provides a checked finite-graph certificate interface; the producer and its input handling remain separate from the Lean decision theorems. For finite-tree answers, graph finiteness is checked separately at the variable roots. Other constructor arities, competing labels, and contravariant arguments likewise require separate semantic transfers.

An explicit top constructor poses a particular obstruction to the present propagation proof: from $X \times Y \leq Z$, the root of Z may be top rather than product. The single-label Horn closure in Section 3 then ceases to express all model choices. Extending the decision theorem to that signature requires a construction that preserves those choices across repeated occurrences.

References

[1] F. Henglein and J. Rehof. The complexity of subtype entailment for simple types. *Proceedings of the 12th annual IEEE symposium on logic in computer science (LICS)*, pp. 352–361, 1997. https://doi.org/10.1109/LICS.1997.614961.

[2] F. Henglein and J. Rehof. Constraint automata and the complexity of recursive subtype entailment. *Automata, languages and programming (ICALP)*, 1443, pp. 616–627, 1998. https://doi.org/10.1007/BFb0055089.

[3] J. Niehren and T. Priesnitz. Non-structural subtype entailment in automata theory. *Information and Computation*, 186(2), pp. 319–354, 2003. https://doi.org/10.1016/S0890-5401(03)00140-8.

[4] J. Niehren and T. Priesnitz. Entailment of non-structural subtype constraints. *Asian computing science conference (ASIAN)*, 1742, pp. 251–265, 1999. https://doi.org/10.1007/3-540-46674-6_22.

[5] Subtype entailment over bottom and product: A self-contained characterization, a corrected reachability lemma, and the border-avoidance problem in occurrence form. 2026. Unpublished memo for Fritz Henglein, 31 August 2026.

[6] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), pp. 323–350, 1977. https://doi.org/10.1137/0206024.

[7] Z. Su, A. Aiken, J. Niehren, T. Priesnitz, and R. Treinen. The first-order theory of subtyping constraints. *ACM Transactions on Programming Languages and Systems*, 2006. https://www.ps.uni-saarland.de/Publications/details/fot:04.html.

[8] F. Breuvart, V. Penelle, and J. Rehof. New results on the non-structural subtype entailment problem. 2019. Research presentation slides. https://lipn.univ-paris13.fr/~breuvart/articles/NSSA_journee.pdf.

[9] A. Schubert. On non-structural subtype entailment. 2021. Withdrawn 20 December 2021; error in Lemma 5.5. https://doi.org/10.48550/arXiv.2109.02458.

[10] Author placeholders. Subtail: Proof and reproduction artifact for bottom/product subtype entailment. 2026. https://github.com/mhenglein/subtail.